

Gossip-Based Protocol for Distributed Data Aggregation

Sistemi Distribuiti e Cloud Computing - Project Report

Federico Scordo

Dipartimento di Ingegneria Informatica
Università degli studi di Roma Tor Vergata

Abstract—Il presente articolo documenta la progettazione e l'implementazione di un sistema distribuito *Gossip-based* specificamente sviluppato per l'aggregazione di dati in reti distribuite. Calcolare metriche globali (quali la somma, la media, il minimo, il massimo e l'estrazione di valori Top-K) è un problema che può essere affrontato seguendo due paradigmi principali: avvalendosi di un nodo coordinatore (approccio centralizzato), oppure in maniera puramente peer-to-peer, senza alcun punto di controllo centrale (approccio decentralizzato). Scegliendo di abbracciare il secondo paradigma per eliminare i Single Point of Failure (SPOF) insiti nel primo approccio, la soluzione proposta combina la disseminazione epidemica asincrona dei messaggi su un protocollo di rete UDP con l'impiego di un'architettura *Multi-CRDT* (Convergent Replicated Data Types di tipo *state-based*). Tramite un aggregatore composito, il sistema calcola simultaneamente tutte le suddette metriche da un'unica struttura dati, veicolata assieme alla topologia di rete mediante tecnica di *piggybacking*. Questa architettura garantisce che l'aggregazione operi in modo commutativo ed idempotente, permettendo al cluster di convergere asintoticamente al medesimo risultato esatto, restando immune alla perdita, alla duplicazione ed ai ritardi asincroni dei pacchetti (*Eventual Consistency*). A supporto del motore matematico, il sistema integra inoltre un meccanismo di failure detection decentralizzato ispirato a SWIM (Scalable Weakly-consistent Infection-style Process Group Membership Protocol), incaricato di isolare dinamicamente i nodi guasti dalla computazione. Il progetto, interamente sviluppato in Go e dotato di un'API di telemetria HTTP per l'osservabilità, è stato validato tramite test end-to-end in memoria, simulazioni di *Traffic Control* e dispiegato su un ambiente cloud multi-nodo containerizzato tramite Docker (AWS Learner Lab), dimostrando la piena tolleranza ai partizionamenti di rete (*split-brain healing*) ed il rientro immediato dei nodi andati in crash (*stateless rejoin*).

I. INTRODUCTION

Nei moderni sistemi distribuiti su larga scala—quali le reti di sensori IoT, le architetture a microservizi ed i cluster di orchestrazione cloud—l'aggregazione continua di dati in tempo reale rappresenta una sfida algoritmica non indifferente. Calcolare dinamicamente metriche globali, come il carico medio di CPU, l'individuazione di picchi anomali (Top-K) o la somma delle risorse disponibili, richiede un coordinamento efficiente tra i nodi partecipanti. Alcune architetture client-server delegano tali computazioni a nodi leader o database centralizzati. Sebbene l'approccio centralizzato offra forti garanzie di sincronizzazione (Strong Consistency), esso introduce inevitabili Single Point of Failure (SPOF) e severi colli di bottiglia prestazionali che degradano rapidamente all'aumentare della scala del sistema.

Per superare i limiti della centralizzazione, si è deciso di sviluppare il progetto adottando un paradigma computazionale puramente simmetrico e decentralizzato. Nelle reti peer-to-peer (P2P), tuttavia, il mantenimento di uno stato globale coerente è limitato da vincoli teorici. In accordo con il Teorema CAP [4] ed il risultato di impossibilità FLP (Fischer, Lynch, Paterson) [5], il sistema proposto sacrifica deliberatamente la consistenza forte (Linearizzabilità) a favore dell'Alta Disponibilità (Availability) e della Tolleranza alle Partizioni (Partition Tolerance). Tale compromesso architetturale assicura che il sistema continui ad operare ed a calcolare stime parziali anche in presenza di interruzioni di rete, abbracciando il modello della Consistenza Eventuale (Eventual Consistency).

Per implementare la disseminazione delle informazioni in un ambiente privo di controllo, l'architettura si avvale di un protocollo epidemico (Gossip) [1]. In questo modello matematico i nodi comunicano in maniera asincrona scambiando iterativamente il proprio stato locale con un sottoinsieme casuale di peer. Tale meccanismo garantisce un'eccellente scalabilità intrinseca, distribuendo equamente il traffico di rete ed eliminando la necessità di orchestrazione topologica.

L'architettura è stata costruita combinando le seguenti metodologie:

- 1) Un motore Gossip asincrono operante su un protocollo di trasporto non affidabile (UDP), scelto per massimizzare il throughput ed azzerando l'overhead dell'handshake TCP.
- 2) L'impiego di un'architettura *Multi-CRDT* (Convergent Replicated Data Types) [3] per modellare ed estrarre **simultaneamente** molteplici funzioni di aggregazione. Condensando lo stato matematico e topologico in un singolo payload tramite *piggybacking*, i CRDT garantiscono matematicamente che le perturbazioni tipiche dell'UDP (duplicazione, perdita e riordinamento dei datagrammi) non alterino la corretta convergenza del calcolo finale.
- 3) L'integrazione di un protocollo di membership distribuito ispirato a SWIM [2], per il monitoraggio vitale dei peer (Failure Detection) e per sopprimere l'impatto dei nodi guasti (churn) sul risultato dell'aggregazione.

L'utilizzo combinato di queste tecnologie assicura che, a dispetto delle anomalie di rete, il calcolo distribuito converga deterministicamente verso il risultato esatto, offrendo un

framework altamente resiliente. I risultati del collaudo in ambiente cloud (AWS Learner Lab), osservati in tempo reale tramite un'API di telemetria HTTP, e le simulazioni di Chaos Engineering (inclusa l'iniezione di latenza tramite *Traffic Control*) hanno certificato l'efficacia della soluzione, tempi di riconvergenza nell'ordine dei millisecondi in scenari di crash dei nodi prolungati e la totale risoluzione autonoma di partizioni di rete (*split-brain*). L'elaborato prosegue strutturando i background teorici (Sezione II), dettagliando la progettazione architetturale (Sezioni III e IV) e, infine, valutando le performance ed i limiti di scalabilità spaziale (Sezioni V e VI).

II. BACKGROUND

A. Epidemic (Gossip) Protocols

I protocolli epidemici [1] rappresentano una classe di algoritmi distribuiti ispirati matematicamente alla diffusione dei virus in popolazioni biologiche. Ad ogni intervallo discreto di tempo Δt (definito *round* o *tick*), ciascun nodo seleziona uniformemente a caso un sottoinsieme di k nodi noti (parametro identificato come *fanout*) ed avvia un'operazione di scambio dati. I protocolli gossip si dividono in tre paradigmi di comunicazione: *Push* (il nodo invia proattivamente il proprio stato ai target), *Pull* (il nodo richiede lo stato dei target) e *Push-Pull* (scambio bidirezionale). In questo sistema è stato adottato un approccio puramente *Push* implementato su protocollo connectionless, al fine di azzerare i tempi di attesa derivanti da chiamate bloccanti. Questa architettura garantisce che l'informazione pervada l'intera rete con un tempo di convergenza logaritmico $O(\log N)$ ed un'elevata resilienza ai fallimenti dei singoli collegamenti fisici. In riferimento alla tassonomia introdotta da Demers et al. [1], nell'architettura implementata lo scambio continuo dell'intero stato topologico ed applicativo si configura come un processo di **Anti-Entropy** piuttosto che di Rumor Mongering. Sebbene quest'ultimo sia più equilibrato in termini di banda, solo l'Anti-Entropy garantisce matematicamente (con probabilità 1) la convergenza totale del cluster, correggendo silenziosamente eventuali corruzioni o partizioni passate.

B. Failure Detection ed il protocollo SWIM

In una rete peer-to-peer priva di un registro centrale, è importante mantenere una vista accurata della topologia (Membership) ed identificare tempestivamente i nodi guasti. Il protocollo SWIM (Scalable Weakly-consistent Infection-style Process Group Membership Protocol) [2] affronta questa limitazione sostituendo i non scalabili heartbeat multicast con un meccanismo di *probing* decentralizzato. Nell'adattamento del modello SWIM usato nel progetto, lo stato di salute di un nodo transita deterministicamente tra Alive, Suspect (sospetto guasto) e Dead (Failed, guasto confermato). Tali transizioni sono guidate da vincoli temporali locali applicati all'istante dell'ultima interazione (LastSeen). In accordo con la letteratura, questa architettura mira a bilanciare la **Strong Completeness** (la garanzia di rilevare ogni crash) con l'**Accuracy** (la minimizzazione dei falsi positivi). A tale scopo

vengono introdotti gli *Incarnation Numbers* (orologi logici), che risolvono due criticità:

- **Gestione dei Falsi Positivi (*Suspicion*)**: Un nodo fortemente rallentato, che sta per essere dichiarato fallito dai peer, può smentire la diceria emettendo un gossip con un incarnation number incrementato, revocando il proprio stato di Suspect.
- **Guarigione e Rejoin**: Un nodo riavviato a seguito di un vero collasso (o isolato da uno *split-brain*) emette un incarnation number strettamente superiore (generato dal timestamp corrente), forzando la rete ad invalidare lo stato Dead in memoria ed a re-includere l'istanza.

C. Convergent Replicated Data Types (CRDT)

Il mantenimento della consistenza forte nei sistemi distribuiti centralizzati richiede l'acquisizione di lock distribuiti o l'impiego di protocolli di consenso gravosi e bloccanti (quali Paxos o Raft). Per evitare colli di bottiglia prestazionali, i database a consistenza eventuale (come Amazon Dynamo o Apache Cassandra) rinunciano ai lock, accettando di dover affrontare i conflitti generati dalle mutazioni di stato concorrenti. Storicamente, tali database gestiscono il problema avvalendosi di *Vector Clocks* per tracciare la relazione di causalità tra le versioni. Tuttavia, i Vector Clocks si limitano a rilevare le collisioni, delegandone spesso la risoluzione ad euristiche deboli e soggette a perdita di dati (come il *Last-Write-Wins*) o scaricando l'onere della riconciliazione semantica direttamente sull'applicazione client. In un'architettura Gossip operante su protocollo inaffidabile (UDP) ad altissima frequenza, le collisioni, le duplicazioni e l'arrivo fuori ordine dei pacchetti costituiscono la norma, non l'eccezione: in un simile contesto, l'approccio tradizionale dei Vector Clocks risulterebbe inefficiente e comprometterebbe l'esattezza dell'aggregazione globale.

Per superare queste limitazioni si è scelto di adottare i CRDT [3], le cui strutture offrono una risoluzione automatica ed asincrona dei conflitti incorporata direttamente nella matematica del dato. La variante utilizzata in questo progetto è classificabile come CvRDT (State-based CRDT). Formalmente, lo spazio degli stati di un CvRDT deve formare un semireticolo superiore (*Join Semilattice* [3]) parzialmente ordinato. La funzione di unione degli stati (Merge), denotata dall'operatore $\sqcup$, si comporta come una mappa di **LWW-Register (Last-Writer-Wins Registers)** indipendenti (uno per ciascuna identità topologica), descritti da Shapiro [3]. Questo costrutto calcola il *Least Upper Bound* tra lo stato locale X e lo stato remoto ricevuto Y introducendo una **relazione d'ordine totale** tramite orologi logici scalari (Epoch e Version incrementali). A differenza dell'applicazione globale del LWW spiegata prima, qui la regola di sovrascrittura si applica esclusivamente all'interno del singolo peer. Questa semantica permette di scartare in modo mirato i pacchetti UDP obsoleti, mantenendo un *footprint* minimale rispetto ai pesanti Vector Clocks e garantendo una convergenza algoritmica automatica e deterministica.

Affinché la convergenza sia matematicamente garantita su una rete inaffidabile, l'operatore di Merge deve obbligatoriamente soddisfare tre regole:

- **Commutatività:** $X \sqcup Y = Y \sqcup X$ (l'ordine di arrivo dei messaggi UDP è influente sulla stima).
- **Associatività:** $(X \sqcup Y) \sqcup Z = X \sqcup (Y \sqcup Z)$.
- **Idempotenza:** $X \sqcup X = X$ (la ricezione accidentale di datagrammi duplicati non altera lo stato).

III. SOLUTION DESIGN

L'architettura del sistema è stata sviluppata interamente in linguaggio Go (Golang): le primitive native di concorrenza offerte dal linguaggio (Goroutines e Channels) sono state sfruttate per orchestrare in modo efficiente e non bloccante i carichi di I/O di rete asincroni, fondamentale in un sistema altamente distribuito. Per prevenire situazioni di *data race* durante il merge concorrente degli stati, l'accesso alla memoria condivisa (come le tabelle di routing) è rigorosamente serializzato tramite lettori/scrittori (`sync.RWMutex`).

Il progetto presenta un'architettura suddivisa in quattro macro-componenti disaccoppiati tramite l'impiego di *Interface*:

- **Transport:** Rappresenta l'astrazione del livello di comunicazione di rete, implementata nell'interfaccia `Transport` (che espone i metodi `Start`, `Send` e `Close`). L'implementazione per l'ambiente di produzione utilizza socket UDP asincrono e procede alla serializzazione dei datagrammi in formato JSON. Tuttavia, per abilitare test di convergenza deterministici end-to-end senza incorrere in *port exhaustion* del sistema operativo locale, è stata progettata una variante *In-Memory Transport* (virtual switch), capace di instradare i byte esclusivamente tramite *Go channels*.
- **Topology:** Gestisce l'identità logica dei peer (`node_id`) e l'evoluzione topologica del cluster. La struct `Manager` gestisce la failure detection e l'aggiornamento della tabella di routing locale. Ogni voce in tabella (struct `Member`) incapsula un endpoint fisico, un `Incarnation Number` intero a 64 bit, il timestamp `LastSeen` ed il ciclo di vita corrente (classificato mediante la enum `Alive`, `Suspect`, `Dead`, `Leave`). Il metodo `CheckTimeouts` si occupa di declassare i peer inattivi e procedere all'eliminazione definitiva superata la soglia di `CleanupTimeout`.
- **Aggregation:** Costituisce il motore logico ed astrae la semantica *CvRDT* tramite l'interfaccia `Aggregator`. Le cinque aggregazioni matematiche (`Sum`, `Average`, `Min`, `Max` e `Top-K`) sono definite da altrettante struct concrete (es. `AverageAggregator`). Il tratto distintivo di tali classi risiede nell'implementazione del metodo `ComputeResult`, il cui algoritmo estrae la metrica filtrando a runtime la mappa dei contributi: i dati appartenenti a nodi non esplicitamente certificati dal modulo `Topology` come attivi o sospetti vengono ignorati, astraendo così dinamicamente l'impatto dei guasti. Per l'architettura *Multi-CRDT*, queste

cinque entità sono orchestrate strutturalmente da un `CompositeAggregator` centrale. Tale componente riceve l'unica mappa in ingresso e, tramite il metodo `ComputeAll`, delega simultaneamente il calcolo alle singole struct sottostanti, estraendo i risultati di tutte le aggregazioni in un singolo passaggio ad ogni iterazione dell'event loop.

- **Core:** La struct `Engine` incorpora e coordina i tre sottomoduli precedenti. L'invocazione di `Start(ctx)` disaccoppia il flusso di esecuzione lanciando un `time.Ticker` all'interno di una goroutine dedicata, istanziando di fatto l'event loop del gossip. Ad ogni tick, la routine innesca `executeRound`, che campiona i target del fanout, assembla i payload combinando stato applicativo e topologia (piggybacking) ed effettua il dispatch su rete. In parallelo, le interruzioni in ricezione innescano la callback asincrona `handleMessage`, che deserializza le strutture, effettua il merge CRDT ed innesca la convergenza della stima locale.

Per completare l'architettura, operano moduli infrastrutturali ausiliari tra cui il package `setup` per il bootstrap dell'applicativo, `message` per la definizione del modello dati e `telemetry` per l'osservabilità esterna tramite HTTP.

IV. SOLUTION DETAILS

A. Application Setup e Lifecycle

Il punto d'ingresso del sistema è nel package `cmd/agent/main.go`. L'inizializzazione del nodo è gestita dal modulo `setup`, il quale effettua il parsing di un file di configurazione YAML tramite la libreria `yaml.v3`, esponendo fallback automatici tramite variabili d'ambiente. Questo approccio permette la portabilità del binario in ambienti containerizzati Docker. La fase di bootstrap procede con l'istanziamento del `Topology Manager`, dell'aggregatore composito (*Multi-CRDT*) ed, infine, dell'Engine core. Oltre al ciclo di gossip, il main lancia due goroutine di background ausiliarie dedicate al controllo dei timeout topologici (`CheckTimeouts`) ed al logging strutturato (`log/slog`), istanziando contestualmente il `TelemetryServer HTTP` per l'esposizione delle metriche.

All'interno del modulo `Topology`, la routine di background `CheckTimeouts` implementa la logica di *Failure Detection* del nodo. L'algoritmo determina le transizioni di stato confrontando periodicamente il timestamp dell'ultima interazione di ciascun peer con i limiti temporali configurati, retrocedendo i nodi inattivi senza la necessità di inviare pacchetti di controllo:

Failure Detection (SWIM)

```
elapsed := now.Sub(member.LastSeen)

if member.Status == StatusAlive && elapsed > m.config.
    SuspectTimeout {
    member.Status = StatusSuspect
} else if member.Status == StatusSuspect && elapsed > (
    m.config.SuspectTimeout + m.config.DeadTimeout) {
    member.Status = StatusDead
    member.DeadSince = now
}
```

Infine, il ciclo di vita intercetta i segnali del sistema operativo (es. SIGTERM, SIGINT) per garantire un *Graceful Shutdown*: prima dell'arresto, il nodo emette attivamente un messaggio `AnnounceLeave` verso i peer e spegne i servizi HTTP in attesa, velocizzando la riconvergenza della rete senza dover subire i ritardi dei timeout.

B. Implementazione del Transport Layer e Concorrenza

L'astrazione del livello di comunicazione di rete è gestita dall'interfaccia `Transport`. L'implementazione utilizzata in ambiente di produzione, `UDPTransport` (localizzata in `internal/transport/udp_transport.go`), gestisce l'intero ciclo di vita delle socket connectionless. L'invocazione del metodo `Start` alloca un `net.ListenUDP` ed avvia in parallelo una goroutine dedicata (metodo `readLoop`). Quest'ultima intercetta i datagrammi in ingresso—gestendo proattivamente i `net.Error` generati dal `SetReadDeadline`—e passa il payload JSON alla callback `MessageHandler`.

Data la concorrenza nel sistema, la sicurezza della memoria (Thread-Safety) all'interno del transport è garantita da primitive native del linguaggio Go: l'arresto controllato del ciclo di lettura (`Close`) è mediato tramite un `sync.WaitGroup`, mentre i cambi di stato interni sono protetti da esecuzioni singole (`sync.Once`) e mutua esclusione (`sync.RWMutex`), assicurando l'assenza di *data race* durante lo shutdown dell'istanza.

La robustezza del ciclo di lettura (`readLoop`) risiede nell'utilizzo combinato del *multiplexing* sui canali (`select`) e dei timeout non bloccanti (`SetReadDeadline`). Questo impedisce alla goroutine di bloccarsi indefinitamente su una `ReadFromUDP`, consentendo l'intercettazione dei segnali di spegnimento (`ctx.Done()` o `t.done`) ed un rilascio pulito delle risorse di rete:

Multiplexing e Socket non bloccanti (UDPTransport)

```
func (t *UDPTransport) readLoop(ctx context.Context) {
    defer t.wg.Done()
    buffer := make([]byte, 65535)

    for {
        select {
            case <-t.done: return
            case <-ctx.Done(): return
            default:
        }

        t.conn.SetReadDeadline(time.Now().Add(250 * time.
            Millisecond))
        n, _, err := t.conn.ReadFromUDP(buffer)
        // ... (gestione degli errori di I/O)

        if t.handler != nil && n > 0 {
            payload := make([]byte, n)
            copy(payload, buffer[:n])
            _ = t.handler(ctx, payload)
        }
    }
}
```

C. Networking e Telemetria

A livello di trasporto, l'architettura sfrutta il protocollo UDP per la disseminazione epidemica. La scelta di UDP tollera la

perdita temporanea dei pacchetti e beneficia dell'assenza di overhead derivante dall'handshake TCP, permettendo l'invio ad alta frequenza degli stati CRDT. Per l'osservabilità esterna, il modulo `telemetry` integra un server HTTP nativo (`TelemetryServer`) atto ad esporre metriche in tempo reale in formato JSON tramite gli endpoint `/health` (per lo stato del processo) e `/metrics`. Quest'ultimo esporta un report completo del nodo: la stima primaria (`estimate`), il contatore dell'event loop (`round`), il numero di peer riconosciuti nella topologia (`known_nodes`) e, soprattutto, l'oggetto composito `all_aggregations` contenente tutti i risultati Multi-CRDT estratti simultaneamente. Essendo il protocollo HTTP basato su TCP, ogni nodo necessita logicamente dell'allocazione di due porte distinte sul sistema operativo host. Per minimizzare la complessità della configurazione e prevenire fastidiose collisioni di porte TCP durante l'esecuzione di test multi-nodo (nella medesima interfaccia di `loopback`), si è deciso di fornire in input allo YAML una singola porta UDP P_{gossip} per il protocollo epidemico, con il server `main` che alloca deterministicamente la porta del server di telemetria TCP traslandola di un offset fisso: $P_{metrics} = P_{gossip} + 1000$. Questo vincolo riduce i parametri configurabili e permette l'assenza di sovrapposizioni a livello socket.

D. Piggybacking

Per ridurre l'overhead di rete ed eludere la frammentazione dei pacchetti UDP, il sistema sfrutta il *piggybacking*. Ad ogni round, nel package `message`, la struct `GossipMessage` viene serializzata in un singolo payload JSON (identificato da un `MessageID` univoco) contenente due entità logiche distinte: il payload matematico (la struct `AggregationState`) ed il digest topologico della rete (un array di `MembershipEntry`). In questo modo, il processo di *Node Discovery* distribuito e l'aggiornamento dell'aggregazione CRDT avvengono contestualmente, riducendo le operazioni I/O del 50%.

Il codice sottostante mostra la struttura dati esatta che viene serializzata ad ogni invio, dove il vettore (`Membership`) viene fuso nello stesso datagramma dello stato applicativo (`State`).

Struttura del Payload con Piggybacking

```
type GossipMessage struct {
    MessageID MessageID      `json:"message_id"`
    SenderID   NodeID                    `json:"sender_id"`
    SentAt     time.Time                 `json:"sent_at"`
    Round      uint64                    `json:"round"`
    State      AggregationState          `json:"state"`
    Membership []MembershipEntry        `json:"membership,
        omitempty"`
}
```

E. Implementazione CRDT Filtrata a Runtime

La semantica CRDT è incapsulata all'interno della struttura `AggregationState`. Questa mantiene in memoria una hash map denominata `Contributions`, indicizzata univocamente per `NodeID`, la quale conserva l'ultimo record unificato (comprensivo dei valori di aggregazione) inviato da ciascun peer della rete.

Per mantenere la monotonicità, la mappa si comporta come una collezione di **LWW-Register (Last-Writer-Wins Registers)** indipendenti, descritti da Shapiro [3]. Il metodo `MergeCRDT` accetta sovrascritture di stato unicamente se il contributo in ingresso possiede un `Epoch` superiore o, a parità di `Epoch`, una `Version` numerica strettamente maggiore rispetto a quella salvata localmente, rigettando eventuali replay-attack o messaggi fuori ordine. La logica di idempotenza e convergenza è garantita dalla condizione di accettazione illustrata nel seguente blocco di codice, dove un contributo remoto sovrascrive quello locale solo se più recente.

Condizione di merge state-based

```
accept := !exists ||
remoteContrib.Epoch > localContrib.Epoch ||
(remoteContrib.Epoch == localContrib.Epoch &&
remoteContrib.Version > localContrib.Version)

if accept {
    merged := Contribution{
        Value: remoteContrib.Value,
        Sum: remoteContrib.Sum,
        Count: remoteContrib.Count,
        Epoch: remoteContrib.Epoch,
        Version: remoteContrib.Version,
    }
    if remoteContrib.TopK != nil {
        merged.TopK = make([]float64, len(remoteContrib.
            TopK))
        copy(merged.TopK, remoteContrib.TopK)
    }
    s.Contributions[nodeID] = merged
    changed = true
}
```

L'architettura Multi-CRDT è implementata tramite il metodo `ComputeAll` dell'aggregatore composito. Come mostrato nel codice sottostante, questo componente aggrega i risultati delegando il calcolo alle rispettive struct concrete:

Calcolo Multi-CRDT

```
func (c *CompositeAggregator) ComputeAll(state *message.
    AggregationState, aliveNodes map[message.NodeID]
    bool) message.AllResults {
    return message.AllResults{
        Sum: c.sum.ComputeResult(state, aliveNodes),
        Average: c.average.ComputeResult(state, aliveNodes),
        Min: c.min.ComputeResult(state, aliveNodes),
        Max: c.max.ComputeResult(state, aliveNodes),
        TopK: c.topK.ComputeTopK(state, aliveNodes),
    }
}
```

Il calcolo effettivo del risultato è delegato al metodo `ComputeResult` delle classi del package `aggregation` (es. `AverageAggregator`). Prima di processare la mappa dei contributi, la funzione interroga il modulo `Topology` per ottenere la lista aggiornata dei nodi considerati `Alive` o `Suspect`. Qualora un nodo sia transitato nello stato `Dead`, il suo contributo matematico—sebbene conservato nella mappa CRDT—viene ignorato a *runtime*.

Questo meccanismo dinamico assicura che, in caso di crash improvviso di un'istanza, la sua metrica venga oscurata, costringendo il cluster superstite a ricollocare la stima globale su un valore accurato e coerente con la nuova topologia del cluster. A livello di codice, tale comportamento è implementato

scartando dinamicamente le chiavi della mappa durante il ciclo computazionale, come evidenziato nel codice sottostante riferito all'aggregazione `Average`.

Filtraggio a runtime dei nodi Dead

```
var totalSum float64
var totalCount uint64
for nodeID, contrib := range state.Contributions {
    // Ignora a runtime i contributi di nodi caduti
    if !aliveNodes[nodeID] {
        continue
    }
    totalSum += contrib.Sum
    totalCount += contrib.Count
}
```

F. Prevenzione dello Split-Brain (Healing)

Una criticità nei protocolli decentralizzati è la partizione di rete (`Split-Brain`). In caso di frammentazione del cluster in due o più sottoreti isolate, i nodi appartenenti a fazioni opposte si dichiareranno a vicenda `Dead` a causa dello scadere dei timeout `SWIM`, convergendo autonomamente verso sotto-aggregati indipendenti.

Per indurre la guarigione della partizione non appena i nodi tornano disponibili, l'istanza implementa un probing stocastico direttamente nel metodo `executeRound`. A prescindere dal normale campionamento casuale del fanout, è stata inserita una regola secondo la quale con una probabilità del 20% (`rand.Float32() < 0.20`), un nodo emette il proprio stato CRDT verso l'indirizzo IP statico dei propri `Seed Peers` originari, anche qualora questi ultimi siano stati catalogati come `Dead` localmente. Questo ping probabilistico funge da sonar esplorativo: al ripristino della connessione tra le due partizioni, i pacchetti forzati raggiungono i seed peer isolati. Poiché i nodi non hanno mai smesso di ticchettare, i loro `Incarnation Numbers` risulteranno enormemente cresciuti. La comparazione di questi ultimi costringerà le rispettive tabelle di membership a soppiantare lo stato `Dead` obsoleto, riunificando la visione topologica ed innescando l'immediato (*merge*) e la riconvergenza dei due sotto-cluster.

Questo approccio è stato implementato nell'Engine del modulo `Core`: una singola istruzione condizionale stocastica previene la disgregazione permanente del cluster con un overhead di rete trascurabile.

Logica stocastica di Split-Brain Healing

```
// Risoluzione partizioni: ping probabilistico (20%)
if len(peers) == 0 || rand.Float32() < 0.20 {
    for _, seedAddr := range e.seedPeers {
        if seedAddr == e.myAddress {
            continue
        }
        _ = e.transport.Send(ctx, seedAddr, payload)
    }
}
```

V. RESULTS AND EVALUATION

Per validare la correttezza architetturale del protocollo Gossip, il sistema è stato sottoposto ad una pipeline di valutazione strutturata in due fasi: test automatizzati end-to-end

(in-memory) e fault-injection applicata ad un deployment cloud reale.

A. Unit ed Integration Tests

L'intera codebase è coperta da unit test (in `tests/aggregation` e `tests/topology`) che verificano l'idempotenza delle aggregazioni ed il cambiamento delle viste topologiche (Membership e Node Discovery). Ma il collaudo principale del sistema risiede nella suite di Integration Test (`tests/integration/cluster_test.go`). Sostituendo il transport UDP con l'`InMemoryTransport` (che veicola i pacchetti interamente tramite Go Channels), è stato possibile allocare cluster di N nodi all'interno del medesimo processo operativo, simulando topologie di rete complesse senza consumare porte del sistema operativo locale. Il controllo sulla raggiungibilità dei peer è gestito da uno switch centrale (`InMemoryBus`). Iniettando a runtime regole di blocco (firewall virtuali), il sistema permette di scartare volontariamente i pacchetti per innescare partizioni silenti. Il seguente estratto mostra la logica del metodo `Send`: oltre allo smistamento condizionale del traffico, l'algoritmo clona fisicamente il payload in transito per scongiurare condizioni di *data race* tra le goroutine riceventi:

Virtual Switch e iniezione di partizioni per Integration Testing

```
func (b *InMemoryBus) Send(ctx context.Context,
    destAddr string, payload []byte) error {
    b.mu.RLock()
    handler, exists := b.handlers[destAddr]
    isBlocked := b.blocked[destAddr]
    b.mu.RUnlock()

    if !exists || isBlocked {
        return nil
    }

    payloadCopy := make([]byte, len(payload))
    copy(payloadCopy, payload)
    go handler(ctx, payloadCopy)
    return nil
}
```

I test condotti in (`cluster_test.go`) dimostrano:

- **Convergenza Matematica:**

Inizializzando nodi con valori disomogenei, il cluster raggiunge costantemente il consenso. Le asserzioni verificano che il delta tra le stime locali ed il valore teorico atteso collassi al di sotto di una tolleranza $\epsilon \leq 10^{-9}$ in pochi millisecondi.

- **Tolleranza ai Crash e Rejoin:**

Arrestare forzatamente l'event loop di un nodo comporta la sua corretta identificazione come Dead da parte dei peer, seguita da un ricalcolo dinamico dell'aggregato. Il riavvio del nodo fallito (stateless) innesca la generazione di un nuovo Incarnation Number, forzando la rete a soppiantare lo stato Dead ed a re-includere l'istanza.

- **Risoluzione Partizioni e Double-Counting:**

L'iniezione malevola di datagrammi ridondanti fallisce nel falsare il conteggio locale, confermando l'idempotenza del CRDT. Al contempo, il blocco del transito dei messaggi tra due metà del cluster induce uno split-brain, il quale

viene puntualmente curato dal meccanismo probabilistico (ping ai seed) non appena si ripristinano i link virtuali.

B. Chaos Engineering

Per testare la robustezza dell'architettura ai crash improvvisi dei nodi, sono stati sviluppati strumenti di Chaos Engineering all'interno della directory `scripts/`. Lo script `demo_crash.sh` orchestra dimostrazioni automatizzate contro i container Docker del cluster, abbattendo (`docker kill`) e riavviando un nodo target. Questa utility interroga simultaneamente gli endpoint HTTP di tutte le repliche, permettendo di misurare istantaneamente l'impatto globale del crash sul tempo di riconvergenza della rete superstita e certificando l'assenza di lock distribuiti pendenti. Per un'osservabilità interattiva, l'utility `fault_dashboard` espone un pannello di controllo a terminale per innescare partizioni manuali spegnendo container specifici in tempo reale, per poi osservare il cambiamento delle metriche HTTP nei server TCP superstiti.

Inoltre, per la valutazione quantitativa del tempo di assestamento asintotico, il framework è stato esteso con lo strumento `scripts/plot_convergence.go`: tale processo interroga periodicamente gli endpoint telemetrici per generare grafici statici che illustrano la traiettoria delle stime locali verso il consenso globale (si veda la Fig. 1).

Oltre alle interruzioni di processo, la robustezza all'asincronia è stata validata sfruttando le primitive di Traffic Control del kernel Linux (`tc qdisc netem`). Applicato il Traffic Control in fase di bootstrap dei container, è stato possibile iniettare latenze crescenti e sfalsate per ogni nodo (da 0 a 10 secondi), simulando un ambiente geografico frammentato per i nodi. I collaudi (`demo_latency.sh`) hanno dimostrato l'assorbimento totale del ritardo asincrono da parte del CRDT e la convergenza della rete.

Di seguito viene mostrato un estratto dei log generati durante una simulazione di rete, evidenziando come l'arresto brutale di un container inneschi i ricalcoli nei nodi superstiti e come il riavvio ripristini istantaneamente lo stato di convergenza.

Estratto log di demo_crash.sh

```
FASE 1 - Stato del Cluster (Convergenza Iniziale)
[OK] node-1 | known: 8 | round: 10 | sum: 360.00 avg:
45.00 topk: [60,70,80]
...
[OK] node-8 | known: 8 | round: 9 | sum: 360.00 avg:
45.00 topk: [60,70,80]

FASE 2 - Hard Crash del Nodo (Simulazione Guasto
Improvviso)
[!] Nodo selezionato per il crash: gossip-node8
Invio segnale SIGKILL (docker kill)
[!] gossip-node8 e' stato terminato forzatamente!

FASE 3 - Reazione del Cluster (Failure Detection SWIM)
Il protocollo SWIM rileverà l'assenza del nodo tramite
timeout.
I nodi vivi ricalcoleranno l'aggregazione escludendo il
nodo morto.

[OK] node-1 | known: 7 | round: 38 | sum: 280.00 avg:
40.00 topk: [50,60,70]
...
[!] Node-8 - NON RAGGIUNGIBILE

FASE 4 - Rejoin del Nodo Crashato
```

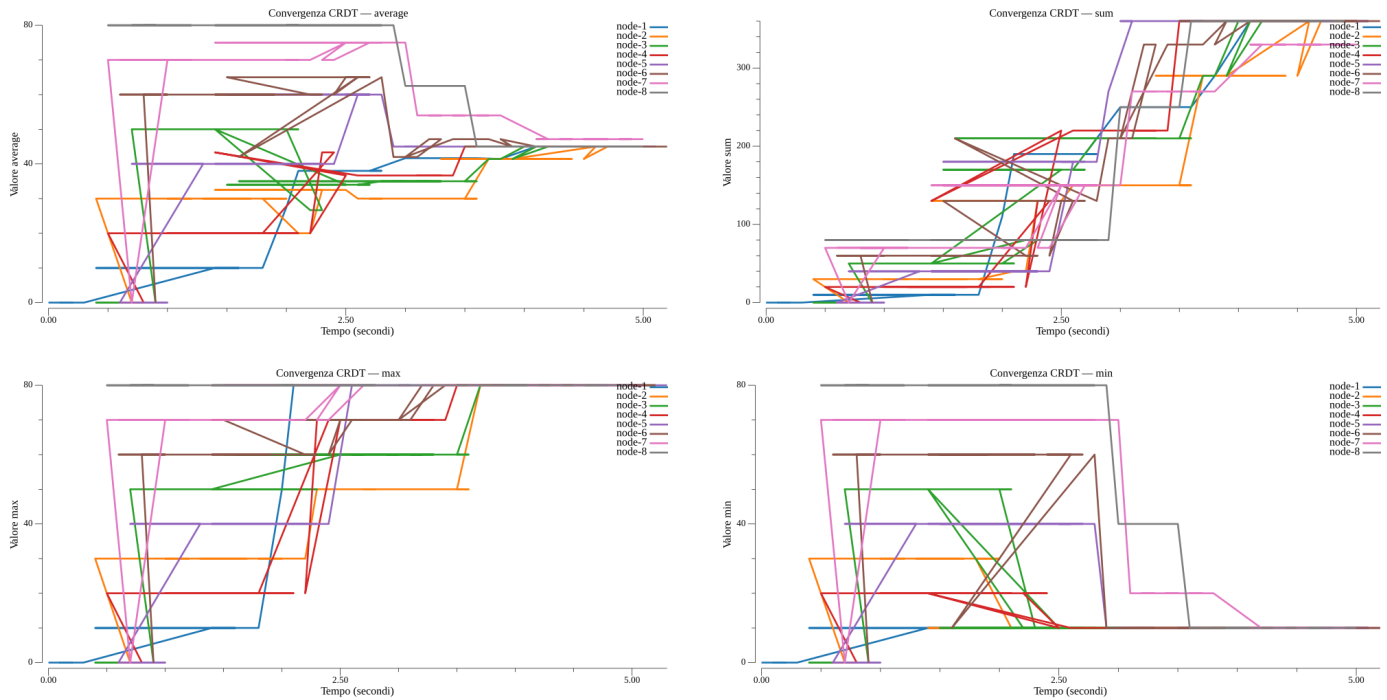


Fig. 1. Traiettorie temporale asintotica verso il consenso globale per le differenti funzioni CRDT.

```
[*] Riavvio di gossip-node8 con docker start...
[OK] gossip-node8 riavviato! Nuovo Incarnation Number
    generato.

FASE 5 - Riconvergenza Completa (Stateless Rejoin)
[OK] node-1 | known: 8 | round: 49 | sum: 360.00 avg:
    45.00 topk: [60,70,80]
...
[OK] node-8 | known: 8 | round: 10 | sum: 360.00 avg:
    45.00 topk: [60,70,80]
```

C. Cloud Deployment (AWS Learner Lab)

Il deployment del sistema in ambiente cloud è stato realizzato sui server Amazon Web Services sfruttando le risorse di AWS Academy Learner Lab. Per testare la flessibilità architetturale del sistema sono stati adottati due approcci distinti.

Docker Compose Multi-Container su Singola Istanza

Una prima validazione è stata condotta istanziando un'unica macchina virtuale Amazon EC2 (Amazon Linux 2023). Sfruttando `docker-compose`, è stato lanciato l'intero cluster di 8 nodi. In questo scenario, Docker genera autonomamente una rete virtuale isolata al cui interno viaggiano i pacchetti Gossip UDP. Il Security Group AWS è stato configurato seguendo il minimo privilegio: il traffico UDP non è stato esposto su Internet, mentre le porte TCP preposte alla telemetria (8001–8008) sono state aperte per permettere l'osservabilità dall'esterno.

Rete P2P Distribuita su Istanze EC2 Separate

Per replicare un ambiente di produzione puramente distribuito, il progetto è stato esteso allocando 5 istanze EC2 distinte, ciascuna ospitante un singolo container. A differenza del caso precedente, i nodi comunicano attraversando la rete fisica del VPC di Amazon, sfruttando gli *IPv4 Privati* delle macchine, iniettati dinamicamente tramite variabili d'ambiente

(SEED_PEERS). Questo setup ha richiesto la riprogettazione dei Security Group AWS, autorizzando esplicitamente il traffico UDP in ingresso (7001–7005) per permettere la propagazione epidemica tra macchine separate.

Interrogando esteriormente gli endpoint `/metrics` tramite richieste HTTP cURL puntate agli IP pubblici delle istanze EC2, si è potuta certificare l'osservabilità del sistema in entrambi gli scenari. I risultati hanno riscontrato una coerenza di stato globale perfetta: interrogando i nodi, ciascuna istanza riporta simultaneamente l'identica topologia ed il medesimo `estimate` convergente, dimostrando la correttezza dell'infrastruttura Gossip-CRDT.

A riprova dell'avvenuto consenso distribuito, di seguito viene mostrata la risposta HTTP ottenuta interrogando la porta 8001 dell'istanza EC2 al termine del bootstrap del cluster (per il primo approccio).

Risposta HTTP su AWS EC2

```
$ curl http://<AWS-IP>:8001/metrics
{
  "node_id": "node-1",
  "primary_aggregation": "average",
  "estimate": 45.0000,
  "known_nodes": 8,
  "round": 8,
  "epoch": 1789226890826009811,
  "uptime_seconds": 9,
  "all_aggregations": {
    "sum": 360.0000,
    "average": 45.0000,
    "min": 10.0000,
    "max": 80.0000,
    "top_k": [60, 70, 80]
  }
}
```

VI. DISCUSSION

A. Verifica Empirica del Teorema CAP

L'implementazione dell'architettura proposta fornisce una dimostrazione del Teorema CAP in contesti P2P. Eliminando il concetto di quorum e le elezioni di un leader, il sistema favorisce l'Availability e la Tolleranza ai Partizionamenti. La conseguenza di questa scelta è il degrado della Linearizzabilità in favore dell'Eventual Consistency. Dai test condotti emerge che, a seguito di un transitorio di rete (es. node churn, packet loss o latenze estreme indotte), i nodi esibiscono temporaneamente stime matematiche discordanti. Tuttavia, la sinergia tra il Gossip e l'idempotenza dei CRDT garantisce che tale divergenza sia un fenomeno esclusivamente transitorio: il sistema culmina nel consenso globale non appena le anomalie si attenuano, onorando le proprietà di *Liveness* della rete.

B. Anomalie di Routing UDP in Ambiente Containerizzato

Durante la validazione tramite l'orchestratore Docker (in locale o su AWS), l'impiego del protocollo UDP ha esposto un limite intrinseco della virtualizzazione di rete. Nelle reti geografiche tradizionali, una partizione silente comporta il drop dei pacchetti senza alcuna notifica al mittente. Al contrario, la disconnessione di un container da una rete virtuale bridge in Docker (per simulare uno split-brain) genera istantaneamente errori di instradamento a livello socket (simili a messaggi *ICMP Destination Unreachable*). Questo comportamento forza le primitive `ReadFromUDP` e `WriteToUDP` di Go in uno stato di errore immediato, corrompendo l'asincronia del protocollo epidemico. Per eludere tale interferenza imposta dal kernel Linux ospite e validare fedelmente gli scenari di churn, gli script di Chaos Engineering sono stati progettati per attuare unicamente arresti a freddo dei processi (tramite *SIGKILL* o `docker stop`). Questo espediente forza il decadimento naturale dei timer `LastSeen` nel failure detector SWIM, permettendo la simulazione della segregazione topologica e della successiva guarigione probabilistica.

C. Limiti di Scalabilità Spaziale e Vincoli di MTU

Una criticità largamente discussa in letteratura riguardante gli State-Based CRDT (CvRDT) riguarda il footprint di memoria e di banda. Nell'architettura implementata, l'aggregazione richiede che l'intera mappa dei contributi cronologici, associata all'identità di tutti gli N nodi del cluster, sia serializzata interamente nel payload JSON ad ogni tick (Piggybacking). Tale criticità spaziale è stata ulteriormente accentuata dall'adozione dell'architettura Multi-CRDT: per calcolare tutte e 5 le metriche simultaneamente, ciascun nodo incapsula nel proprio payload un record unificato comprensivo di Somma, Conteggio, Valore scalare e di un intero array allocato per i Top-K. Di conseguenza, il consumo di heap RAM e la dimensione del datagramma crescono linearmente $O(N)$. Sebbene l'implementazione si sia rivelata estremamente efficiente per cluster limitati, all'aumentare esponenziale dei peer il volume in byte del pacchetto è destinato a collidere con la Maximum Transmission Unit (MTU) dei frame Ethernet (fissata convenzionalmente a 1500 byte). Il superamento di

tale soglia comporterebbe una frammentazione a livello IP, incrementando drasticamente le probabilità di corruzione e drop dei datagrammi e degradando la velocità teorica di convergenza $O(\log N)$.

VII. CONCLUSIONS

Questo progetto si è concentrato sull'implementazione e sull'analisi di un'architettura decentralizzata basata su un motore Gossip, validandone le dinamiche per il calcolo di metriche aggregate in ambienti paritetici. I risultati raccolti confermano che l'assenza di un nodo coordinatore (SPOF) non preclude il raggiungimento di uno stato globale accurato.

La combinazione tra strutture dati State-Based CRDT ed un meccanismo di Failure Detection distribuito ispirato a SWIM ha permesso di immunizzare l'aggregazione dalla volatilità dei nodi (churn) e dall'intrinseca inaffidabilità del mezzo trasmissivo UDP. Le pipeline di collaudo ed il deployment su istanze cloud EC2 hanno validato sul campo le capacità di *split-brain healing*, di *stateless rejoin* e l'immunità ai ritardi geografici asincroni, sopprimendo la necessità di introdurre algoritmi di consenso gravosi e bloccanti (quali Raft o Paxos) o fragili meccanismi di Leader Election.

Nonostante il successo operativo del framework, l'analisi dei colli di bottiglia spaziali documentata nella sezione precedente apre la strada a migliorie per massimizzare la scalabilità orizzontale. In primo luogo, l'architettura potrebbe essere estesa ottimizzando la propagazione degli stati CRDT, permettendo ai nodi di trasmettere in rete unicamente i differenziali degli stati mutati rispetto all'ultimo round, comprimendo drasticamente l'impatto sulla banda passante. In secondo luogo, il traffico topologico in piggybacking potrebbe essere alleggerito sostituendo l'elenco in chiaro dei peer con strutture dati probabilistiche ultraleggere, minimizzando il costo marginale della Node Discovery. Infine, per abilitare l'esecuzione sicura in reti geografiche pubbliche, il motore necessiterebbe di un potenziamento tramite protocolli crittografici e di Tolleranza ai Guasti Bizantini, al fine di proteggere il calcolo distribuito dalle manomissioni di peer malevoli. L'integrazione di queste migliorie permetterebbe al sistema di scalare agilmente verso la gestione di decine di migliaia di agenti in scenari di Edge Computing ed IoT.

REFERENCES

- [1] A. Demers et al., "Epidemic algorithms for replicated database maintenance," *Proceedings of the sixth annual ACM Symposium on Principles of distributed computing*, 1987.
- [2] A. Das, I. Gupta, and A. Motivala, "SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol," *IEEE International Conference on Dependable Systems and Networks*, 2002.
- [3] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "A comprehensive study of Convergent and Commutative Replicated Data Types," *INRIA Research Report 7506*, 2011.
- [4] (Teorema CAP): S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services", *ACM SIGACT News*, vol. 33, no. 2, pp. 51-59, 2002.
- [5] (Teorema FLP): M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process", *Journal of the ACM (JACM)*, vol. 32, no. 2, pp. 374-382, 1985.